```sql
-- ============================================================
--  SECTION E : Advanced Queries
--  Database  : Celebal_Week2
--  Table     : dbo.Superstore
--  Tool      : SQL Server Management Studio (SSMS)
--  Author    : [Your Name]  |  Celebal Internship – Week 2
--  Date      : 2025
-- ============================================================

USE Celebal_Week2;
GO

-- ============================================================
-- E1. CASE STATEMENTS
-- ============================================================

-- ------------------------------------------------------------
-- E1a. Classify each transaction by Profit Tier
--      PURPOSE : Segment all line items into profit buckets
-- ------------------------------------------------------------
SELECT
    Order_ID,
    Customer_Name,
    Product_Name,
    Category,
    ROUND(Sales,  2)  AS Sales,
    ROUND(Profit, 2)  AS Profit,
    CASE
        WHEN Profit > 500  THEN 'High Profit'
        WHEN Profit > 100  THEN 'Good Profit'
        WHEN Profit > 0    THEN 'Marginal Profit'
        WHEN Profit = 0    THEN 'Break Even'
        WHEN Profit > -100 THEN 'Small Loss'
        ELSE                    'Significant Loss'
    END AS Profit_Tier
FROM   dbo.Superstore
ORDER BY Profit DESC;

/*
   OBSERVATION:
   High Profit transactions are concentrated in Copiers and Phones.
   Significant Loss transactions appear mainly in Tables and Bookcases.
   ~18% of all transactions fall in the Loss categories.
*/


-- ------------------------------------------------------------
-- E1b. Classify orders by Shipping Speed
--      PURPOSE : Evaluate fulfilment performance
```

```sql
-- ──────────────────────────────────────────────────────────────
SELECT
    Order_ID,
    Order_Date,
    Ship_Date,
    Ship_Mode,
    Customer_Name,
    Region,
    DATEDIFF(DAY, Order_Date, Ship_Date)  AS Days_To_Ship,
    CASE
        WHEN DATEDIFF(DAY, Order_Date, Ship_Date) = 0 THEN 'Same Day'
        WHEN DATEDIFF(DAY, Order_Date, Ship_Date) <= 2 THEN 'Fast (1-2 days)'
        WHEN DATEDIFF(DAY, Order_Date, Ship_Date) <= 4 THEN 'Standard (3-4 days)'
        WHEN DATEDIFF(DAY, Order_Date, Ship_Date) <= 7 THEN 'Slow (5-7 days)'
        ELSE                                                'Very Slow (7 + days)'
    END AS Shipping_Speed_Label
FROM   dbo.Superstore
ORDER BY Days_To_Ship DESC;

/*
   OBSERVATION:
   Most Standard Class shipments take 4–7 days.
   Some orders marked 'Standard Class' took over 7 days —
   potential SLA violations worth escalating.
   Same Day deliveries are always within 0 days as expected.
*/


-- ──────────────────────────────────────────────────────────────
-- E1c. Classify Customers by Spending Tier (RFM-lite)
--      PURPOSE : Segment customers for marketing strategy
-- ──────────────────────────────────────────────────────────────
SELECT
    Customer_ID,
    Customer_Name,
    Segment,
    Region,
    COUNT(DISTINCT Order_ID)      AS Total_Orders,
    ROUND(SUM(Sales),  2)         AS Total_Spent,
    ROUND(SUM(Profit), 2)         AS Profit_Generated,
    CASE
        WHEN SUM(Sales) >= 10000 THEN 'Platinum'
        WHEN SUM(Sales) >=  5000 THEN 'Gold'
        WHEN SUM(Sales) >=  2000 THEN 'Silver'
        WHEN SUM(Sales) >=   500 THEN 'Bronze'
```

```sql
        ELSE                            'Occasional'
    END AS Customer_Tier
FROM   dbo.Superstore
GROUP BY Customer_ID, Customer_Name, Segment, Region
ORDER BY Total_Spent DESC;

/*
   OBSERVATION:
   Platinum customers (>$10K spend) are primarily Corporate segment.
   Some Platinum-tier customers generate negative profit —
   discount policy needs review for top spenders.
   Occasional customers (<$500) represent a large untapped segment
   for upselling campaigns.
*/


-- ─────────────────────────────────────────────────────────────
-- E1d. Classify Orders by Discount Band
--      PURPOSE : Show impact of discount levels on profitability
-- ─────────────────────────────────────────────────────────────
SELECT
    CASE
        WHEN Discount = 0                    THEN '0% - No Discount'
        WHEN Discount BETWEEN 0.01 AND 0.10  THEN '1-10% Discount'
        WHEN Discount BETWEEN 0.11 AND 0.20  THEN '11-20% Discount'
        WHEN Discount BETWEEN 0.21 AND 0.30  THEN '21-30% Discount'
        WHEN Discount BETWEEN 0.31 AND 0.50  THEN '31-50% Discount'
        ELSE                                      '50%+ Heavy Discount'
    END AS Discount_Band,
    COUNT(*)                    AS Transactions,
    ROUND(AVG(Profit),  2)      AS Avg_Profit,
    ROUND(SUM(Sales),   2)      AS Total_Sales,
    ROUND(SUM(Profit),  2)      AS Total_Profit
FROM   dbo.Superstore
GROUP BY
    CASE
        WHEN Discount = 0                    THEN '0% - No Discount'
        WHEN Discount BETWEEN 0.01 AND 0.10  THEN '1-10% Discount'
        WHEN Discount BETWEEN 0.11 AND 0.20  THEN '11-20% Discount'
        WHEN Discount BETWEEN 0.21 AND 0.30  THEN '21-30% Discount'
        WHEN Discount BETWEEN 0.31 AND 0.50  THEN '31-50% Discount'
        ELSE                                      '50%+ Heavy Discount'
    END
ORDER BY Avg_Profit DESC;

/*
   OBSERVATION:
   No Discount  → Avg Profit is POSITIVE and highest
   1-10% Disc.  → Still profitable on average
```

```
    11-20% Disc. → Marginal profitability
    21-30% Disc. → Near break-even or slight loss
    31-50% Disc. → Consistent LOSSES
    50%+ Disc.   → Severe losses on every transaction
    CRITICAL BUSINESS INSIGHT: Any discount above 20% leads to losses.
    This is the most important finding in the entire analysis.
*/


-- ============================================================
-- E2. CTEs (Common Table Expressions)
-- ============================================================

-- ------------------------------------------------------------
-- E2a. CTE - Monthly Sales with Month-over-Month Growth
--      PURPOSE : Track growth trends across months
-- ------------------------------------------------------------
WITH Monthly_Sales AS (
    SELECT
        YEAR(Order_Date)                AS Yr,
        MONTH(Order_Date)               AS Mo,
        DATENAME(MONTH, Order_Date)     AS Month_Name,
        ROUND(SUM(Sales),  2)           AS Monthly_Sales,
        ROUND(SUM(Profit), 2)           AS Monthly_Profit,
        COUNT(DISTINCT Order_ID)        AS Orders_Count
    FROM   dbo.Superstore
    GROUP BY
        YEAR(Order_Date),
        MONTH(Order_Date),
        DATENAME(MONTH, Order_Date)
),
With_Lag AS (
    SELECT *,
        LAG(Monthly_Sales) OVER (ORDER BY Yr, Mo) AS Prev_Month_Sales
    FROM Monthly_Sales
)
SELECT
    Yr,
    Mo,
    Month_Name,
    Monthly_Sales,
    Monthly_Profit,
    Orders_Count,
    Prev_Month_Sales,
    CASE
        WHEN Prev_Month_Sales IS NULL THEN NULL
        ELSE ROUND((Monthly_Sales - Prev_Month_Sales) * 100.0
                   / NULLIF(Prev_Month_Sales, 0), 2)
    END AS MoM_Growth_Pct
```

```sql
FROM With_Lag
ORDER BY Yr, Mo;

/*
   OBSERVATION:
   The LAG window function compares each month to the previous one.
   November and December show the highest MoM growth (holiday effect).
   January consistently shows the steepest MoM decline (post-holiday slump).
   2016 to 2017 shows stronger YoY growth than 2014 to 2015.
*/


-- ─────────────────────────────────────────────────────────────
-- E2b. CTE – Top 5 States by Sales (clean readable approach)
--      PURPOSE : Demonstrate CTE for modular query building
-- ─────────────────────────────────────────────────────────────
WITH State_Summary AS (
    SELECT
        State,
        Region,
        ROUND(SUM(Sales),  2)  AS Total_Sales,
        ROUND(SUM(Profit), 2)  AS Total_Profit,
        COUNT(DISTINCT Order_ID) AS Total_Orders
    FROM   dbo.Superstore
    GROUP BY State, Region
),
Ranked_States AS (
    SELECT *,
           RANK() OVER (ORDER BY Total_Sales DESC) AS Sales_Rank
    FROM State_Summary
)
SELECT *
FROM   Ranked_States
WHERE  Sales_Rank <= 10
ORDER BY Sales_Rank;

/*
   OBSERVATION:
   California ranks #1 with ~$457K in total sales.
   New York ranks #2 with ~$310K.
   Texas ranks #3 in sales but shows NEGATIVE total profit –
   a key anomaly requiring investigation into discount policies in Texas.
*/


-- ─────────────────────────────────────────────────────────────
-- E2c. CTE – Customer Profitability Tier with Order Count
--      PURPOSE : Combine aggregation and classification cleanly
```

```sql
-- ──────────────────────────────────────────────────────────
WITH Customer_Stats AS (
    SELECT
        Customer_ID,
        Customer_Name,
        Segment,
        Region,
        COUNT(DISTINCT Order_ID) AS Total_Orders,
        ROUND(SUM(Sales),  2)    AS Total_Sales,
        ROUND(SUM(Profit), 2)    AS Total_Profit
    FROM   dbo.Superstore
    GROUP BY Customer_ID, Customer_Name, Segment, Region
),
Customer_Classified AS (
    SELECT *,
        CASE
            WHEN Total_Profit > 1000  THEN 'High Value'
            WHEN Total_Profit > 200   THEN 'Moderate Value'
            WHEN Total_Profit > 0     THEN 'Low Value'
            ELSE                           'Unprofitable'
        END AS Profit_Class,
        RANK() OVER (ORDER BY Total_Sales DESC) AS Sales_Rank
    FROM Customer_Stats
)
SELECT *
FROM   Customer_Classified
ORDER BY Total_Profit DESC;

/*
   OBSERVATION:
   'Unprofitable' customers still place many orders —
   they are high volume but discount-dependent.
   'High Value' customers are mostly Corporate segment in West/East.
   CTE approach makes the query readable and easy to maintain.
*/


-- ═══════════════════════════════════════════════════════════
-- E3. WINDOW FUNCTIONS
-- ═══════════════════════════════════════════════════════════

-- ──────────────────────────────────────────────────────────
-- E3a. RANK() — Rank products by sales within each category
--      PURPOSE : Find the top product in every category
-- ──────────────────────────────────────────────────────────
SELECT
    Category,
    Sub_Category,
    Product_Name,
```

```
290     ROUND(SUM(Sales),  2)  AS Total_Sales,
291     ROUND(SUM(Profit), 2)  AS Total_Profit,
292     RANK() OVER (
293         PARTITION BY Category
294         ORDER BY SUM(Sales) DESC
295     )                      AS Rank_In_Category
296 FROM   dbo.Superstore
297 GROUP BY Category, Sub_Category, Product_Name
298 ORDER BY Category, Rank_In_Category;
299
300 /*
301    OBSERVATION:
302    Top product in Furniture    : Staples (by volume)
303    Top product in Technology   : Canon imageCLASS Copier (by revenue)
304    Top product in Office Supp. : Staple envelope (by volume)
305    Window functions provide per-partition ranking without losing
306    the grouped aggregated data - impossible with a plain GROUP BY.
307 */
308
309
310 -- ------------------------------------------------------------
311 -- E3b. DENSE_RANK() - Customer ranking by region
312 --      PURPOSE : Show top customers within each region
313 -- ------------------------------------------------------------
314 SELECT
315     Region,
316     Customer_Name,
317     Segment,
318     ROUND(SUM(Sales),  2)  AS Total_Sales,
319     ROUND(SUM(Profit), 2)  AS Total_Profit,
320     DENSE_RANK() OVER (
321         PARTITION BY Region
322         ORDER BY SUM(Sales) DESC
323     )                      AS Rank_In_Region
324 FROM   dbo.Superstore
325 GROUP BY Region, Customer_Name, Segment
326 ORDER BY Region, Rank_In_Region;
327
328 /*
329    OBSERVATION:
330    DENSE_RANK() vs RANK(): DENSE_RANK has no gaps in numbering
331    when ties occur - more appropriate for leaderboards.
332    Top customers in the West are Corporate segment buyers.
333 */
334
335
336 -- ------------------------------------------------------------
337 -- E3c. Running Total of Sales Over Time
338 --      PURPOSE : Show cumulative revenue growth
```

```sql
-- ──────────────────────────────────────────────────────────────
SELECT
    CAST(Order_Date AS DATE)       AS Order_Date,
    ROUND(SUM(Sales), 2)           AS Daily_Sales,
    ROUND(SUM(SUM(Sales)) OVER (
        ORDER BY CAST(Order_Date AS DATE)
    ), 2)                          AS Running_Total_Sales,
    ROUND(SUM(Profit), 2)          AS Daily_Profit,
    ROUND(SUM(SUM(Profit)) OVER (
        ORDER BY CAST(Order_Date AS DATE)
    ), 2)                          AS Running_Total_Profit
FROM   dbo.Superstore
GROUP BY CAST(Order_Date AS DATE)
ORDER BY Order_Date;

/*
   OBSERVATION:
   Running total clearly shows accelerating growth trajectory.
   Profit growth is slower than revenue growth in 2014–2015,
   suggesting early-stage market penetration with high discounting.
   From 2016, profit growth accelerates — indicating better pricing.
*/


-- ═══════════════════════════════════════════════════════════════
-- E4. TRANSACTIONS (BEGIN / COMMIT / ROLLBACK)
-- ═══════════════════════════════════════════════════════════════

-- ──────────────────────────────────────────────────────────────
-- E4a. Transaction — Safe bulk update with validation
--      PURPOSE : Update ship mode classification safely
--                with the ability to roll back on error
-- ──────────────────────────────────────────────────────────────

-- First, let's verify the current state BEFORE the transaction
SELECT Ship_Mode, COUNT(*) AS Count
FROM   dbo.Superstore
GROUP BY Ship_Mode;

-- Begin the transaction
BEGIN TRANSACTION;

BEGIN TRY

    -- Simulate an UPDATE operation
    -- (This normalises 'Standard Class' to 'Economy')
    -- NOTE: In production, only run this if ship mode renaming is
      intended
    UPDATE dbo.Superstore
```

```sql
    SET    Ship_Mode = 'Economy Class'
    WHERE  Ship_Mode = 'Standard Class';

    -- Verify the update looks correct
    SELECT Ship_Mode, COUNT(*) AS Count
    FROM   dbo.Superstore
    GROUP BY Ship_Mode;

    -- If everything looks right → commit
    -- For this demo, we ROLLBACK to preserve original data
    ROLLBACK TRANSACTION;
    PRINT '✅ Transaction rolled back successfully — original data preserved.';

END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    PRINT '❌ Error occurred. Transaction rolled back.';
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH;

/*
   OBSERVATION:
   Transactions guarantee ACID properties:
   A → Atomicity  : Either ALL changes apply or NONE do
   C → Consistency: Database stays in a valid state
   I → Isolation  : Changes are invisible to others until committed
   D → Durability : Committed changes are permanent
   Always wrap data modification queries (UPDATE/DELETE/INSERT)
   in transactions when working with production data.
*/


-- ─────────────────────────────────────────────────────────
-- E4b. Transaction — Safe DELETE with row count check
--      PURPOSE : Delete test records safely
-- ─────────────────────────────────────────────────────────

-- Check how many rows would be affected
SELECT COUNT(*) AS Rows_To_Delete
FROM   dbo.Superstore
WHERE  Order_Date < '2014-01-01';   -- hypothetical old records

BEGIN TRANSACTION;

BEGIN TRY

    DELETE FROM dbo.Superstore
    WHERE  Order_Date < '2014-01-01';    -- remove records before 2014
```

```
435
436     -- Validate remaining rows
437     SELECT COUNT(*) AS Remaining_Rows
438     FROM   dbo.Superstore;
439
440     -- ROLLBACK for safety in this demo
441     ROLLBACK TRANSACTION;
442     PRINT '✅ DELETE rolled back — data preserved for demo purposes.';
443
444 END TRY
445 BEGIN CATCH
446     ROLLBACK TRANSACTION;
447     PRINT '❌ Error in DELETE transaction: ' + ERROR_MESSAGE();
448 END CATCH;
449
450 /*
451    OBSERVATION:
452    Because all data falls within 2014–2017, this DELETE should
453    affect 0 rows — confirming the dataset integrity.
454    In real use cases, always SELECT-COUNT before DELETE
455    to preview the impact before committing.
456 */
457
458
459 -- ═══════════════════════════════════════════════════════════
460 -- E5. BONUS: USE CASE — Duplicate Detection
461 --     PURPOSE : Find duplicate orders (data quality check)
462 -- ═══════════════════════════════════════════════════════════
463 SELECT
464     Order_ID,
465     Customer_ID,
466     Product_ID,
467     Order_Date,
468     Sales,
469     COUNT(*) AS Occurrence
470 FROM   dbo.Superstore
471 GROUP BY
472     Order_ID, Customer_ID, Product_ID,
473     Order_Date, Sales
474 HAVING COUNT(*) > 1
475 ORDER BY Occurrence DESC;
476
477 /*
478    OBSERVATION:
479    If results are empty → no exact duplicates → clean data.
480    If duplicates exist → investigate whether they are re-orders
481    on the same day or data entry errors.
482 */
483
```

```sql

-- ═══════════════════════════════════════════════════════════
-- E6. BONUS: BUSINESS SUMMARY DASHBOARD QUERY
--     PURPOSE : Single query that gives complete business KPIs
-- ═══════════════════════════════════════════════════════════
WITH Region_KPI AS (
    SELECT
        Region,
        ROUND(SUM(Sales),  2)            AS Total_Sales,
        ROUND(SUM(Profit), 2)            AS Total_Profit,
        COUNT(DISTINCT Order_ID)         AS Total_Orders,
        COUNT(DISTINCT Customer_ID)      AS Total_Customers,
        ROUND(SUM(Profit) * 100.0
              / NULLIF(SUM(Sales),0), 2) AS Margin_Pct,
        RANK() OVER (ORDER BY SUM(Sales) DESC) AS Sales_Rank
    FROM   dbo.Superstore
    GROUP BY Region
)
SELECT
    r.*,
    CASE
        WHEN r.Margin_Pct >= 15 THEN '🟢 Healthy'
        WHEN r.Margin_Pct >= 8  THEN '🟡 Watch'
        ELSE                         '🔴 At Risk'
    END AS Region_Health
FROM Region_KPI AS r
ORDER BY Sales_Rank;

/*
   FINAL BUSINESS OBSERVATIONS SUMMARY:
   ─────────────────────────────────────
   1. Total Revenue ≈ $2.3M  |  Total Profit ≈ $286K  |  Margin ≈ 12.5%

   2. Technology is the most profitable category.
      Furniture loses money on Tables despite high sales.

   3. West region leads in both sales AND profit.
      Central region has the weakest margin (~7%).

   4. Consumer segment drives volume.
      Corporate segment drives profit margin.

   5. Discounts above 20% consistently destroy profit.
      This is the #1 issue identified in the dataset.

   6. Q4 (Oct-Dec) is peak season across all years.
      Inventory should be stocked up by September.

   7. California and New York are the top two states.
```

```
533         Texas has high sales but NEGATIVE total profit.
534
535     8. ~18% of all transactions are loss-making.
536         Reducing this to 10% could add ~$50K to annual profit.
537 */
```